МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Построение и анализ алгоритмов»

Тема: Алгоритм Кнута-Морриса-Пратта

Студент гр. 4343 _____ Едыгеев Р. Б.

Преподаватель _____ Калишенко Е. Л.

Санкт-Петербург

2026

Цель работы.

Необходимо реализовать программу, использующую алгоритм Кнута-Морриса-Пратта для поиска подстроки в строке.

Задание 1.

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P (|P|≤25000) и текста T (|T|≤5000000) найдите все вхождения P в T.

Вход:

- Первая строка – P;
- Вторая строка – T;

Выход:

Индексы начал вхождений P в T, разделённые запятой; если P не входит в T, то вывести -1.

Задание 2.

Заданы две строки A (|A|≤5000000) и B (|B|≤5000000). Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B, склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка – A;
- Вторая строка – B;

Выход:

Если A является циклическим сдвигом B, то индекс начала строки B в A; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Основные теоретические положения.

Алгоритм Кнута–Морриса–Пратта (КМП) — это эффективный метод поиска подстроки в строке, работающий за линейное время O(|P| + |T|), где P — образец (pattern), а T — текст. Ключевая идея алгоритма заключается в

использовании предварительно вычисленной префикс-функции, которая позволяет избежать повторных сравнений символов при несовпадении.

Префикс-функция π(i) для строки P определяется как длина наибольшего собственного префикса подстроки P[0..i], который одновременно является её суффиксом. Собственный префикс означает, что он не равен всей подстроке. Например, для строки "abacab" префикс-функция даёт массив [0, 0, 1, 0, 1, 2]: у подстроки "aba" префикс "a" равен суффиксу "a" (длина 1), у "abacab" префикс "ab" равен суффиксу "ab" (длина 2). Вычисление префикс-функции само по себе выполняется за линейное время с помощью двух указателей: один указывает на текущую позицию i в строке, а второй (j) отслеживает длину текущего совпадающего префикс-суффикса. При несовпадении символов указатель j откатывается не в начало, а к значению π(j-1), что гарантирует линейную сложность.

Сам поиск подстроки работает следующим образом: указатель i движется по тексту T слева направо, а указатель j отслеживает, сколько символов образца P уже совпало с текстом. Если символы text[i] и pattern[j] совпадают, j увеличивается. Если j достигает длины образца, значит найдено вхождение, начинающееся с индекса i - |P| + 1. При несовпадении алгоритм не сбрасывает j в ноль и не возвращает i назад, а присваивает j = π(j-1), эффективно сдвигая образец вправо на минимально необходимое расстояние, используя информацию о структуре самого образца. После нахождения вхождения j также присваивается значение π(j-1) для продолжения поиска перекрывающихся вхождений. Благодаря этому каждая позиция в тексте обрабатывается ровно один раз, а каждый откат уменьшает j, но общее количество откатов не превышает количества продвижений вперёд, что и даёт линейное время работы.

Циклическим сдвигом строки B называется строка A, которую можно получить, разрезав B на две части и поменяв их местами: A = B[k:] + B[:k] для некоторого k. Задача определения, является ли строка A циклическим сдвигом строки B, сводится к поиску B как подстроки в удвоенной строке A + A.

Например, для A = "defabc" и B = "abcdef" удвоение даёт "defabcdefabc", и B входит в неё с индекса 3, что означает, что A — это B, сдвинутая на 3 позиции. Чтобы не хранить удвоенную строку A, часто перемещаются по A при помощи деления инкремента на длину строки A с остатком (I mod len(A)).

Выполнение работы.

Для решения задачи была разработана программа на языке Python (см. Приложение А).

Пайплайн работы алгоритма.

Программа имеет возможность быть запущенной для решения задачи поиска подстроки в строке (без использования флагов), а также для поиска циклического сдвига (с флагом -c). Также есть вывод справки программы (флаг -h).

В обоих случаях для начала запускается расчёт префикс-функции для строки (Паттерна).

Поиск подстроки. Получив массив pi, функция kmp_search запускает поиск подстроки P в строке T. Затем инициализируется счётчик j нулём и заводится пустой список result. В цикле по индексу i от нуля до конца текста происходит основная работа: на каждой итерации в файл записывается текущее значение i, после чего выполняется цикл отката: пока j больше нуля и символ текста text[i] не совпадает с символом образца pattern[j], выводится информация о несовпадении и j присваивается значение pi[j-1]. Если символы совпадают, j увеличивается. Если после этого j сравнивается с длиной образца, это означает, что найдено очередное вхождение. Тогда вычисленный индекс i - n + 1 добавляется в список result, в файл записывается строка о найденном результате, а счётчик j откатывается к pi[j-1] для продолжения поиска возможных перекрывающихся вхождений. Если вхождений нет, печатается -1, иначе индексы выводятся через запятую.

Циклический сдвиг. Программа вызывает функцию cyclic_shift_search. Далее проверяется равенство длин строк: если они не совпадают, пишется сообщение об ошибке, возвращается -1. Для пустых строк возвращается 0. Поиск циклического сдвига сводится к поиску B в виртуально удвоенной строке A+A, поэтому заводится счётчик j равный нулю и запускается цикл по i от нуля до 2n-1, где n — длина строк. На каждом шаге из A берётся символ с индексом i по модулю n, что эмулирует обращение к удвоенной строке без её физического создания. Происходит откат j при несовпадении текущего символа с B[j] и продвижение вперёд при совпадении. Если j достигает длины строки B, это означает, что в виртуальной удвоенной строке найдено вхождение B, и функция записывает в ответ сообщение о завершении и вычисленный индекс начала вхождения i - n + 1. Если цикл завершился, а вхождение так и не было найдено, файл закрывается и возвращается -1.

Программа выводит отладочную информацию в файл “out.txt”.

Анализ сложности алгоритма.

Расчет префикс-функции. Внешний цикл for(1, … n-1) и внутренний цикл while() дают в совокупности O(n), т.к. внешний цикл проходит по всей строке и может увеличить величину j не более, чем на n – 1 (т.к. за каждую итерацию при совпадении символов j увеличивается на 1). Внутри цикла while переменная j только уменьшается при каждом выполнении тела цикла, причём каждый раз минимум на 1 (через присваивание j = pi[j-1], которое всегда меньше текущего j). Поскольку j не может уйти в минус и изначально равна 0, общее количество уменьшений j за всё время работы алгоритма не может превысить общего количества увеличений j, то есть не более n−1 раз. Внешний цикл дает n итераций, внутренний суммарно выполнится не более n раз, итого O(n + n) = O(n).

Поиск подстроки. Подход аналогичен построению префикс функции. С учетом ее построения, алгоритм имеет сложность O(n + m), где m – длина второй строки.

Циклический сдвиг. Внешний цикл итерируется от 0 до 2n – 1, в основном работает по тому же принципу, значит имеет сложность O(2n + n) = O(n).

Сложность по памяти. Хранится три строки – две входные и префикс-функция. В алгоритме циклического сдвига строки не складываются, а происходит проход по строке посредством деления инкремента на длину строки с остатком. Следовательно, сложность – O(n).

Тестирование.

Для тестирования были написаны следующие тесты, покрывающие граничные случаи:

1. Проверка на некорректный ввод, а именно ввод пустых строк (Рис. 1 - 2):

```
crakeny@LAPTOP-TQ9SBN6I:/mnt/c/Users/
c$ python3 KMP.py


EMPTY STRINGS
```

Рисунок 1. Ввод пустых строк

```
crakeny@LAPTOP-TQ9SBN6I:/mnt/c/Users/Hp

helloworld
EMPTY STRINGS
```

Рисунок 2. Ввод одной пустой строки

2. Штатный ввод (Алгоритм поиска подстроки) (Рис. 3):

```
crakeny@LAPTOP-TQ9SBN6I:/mnt/c/Users/Hp/OneDrive/Рабочий стол/projects/2_COURSE/BAA
ful
Oh the weather outside is frightful, but the fire is so delightful
32,63
```

Рисунок 3. Демонстрация работы поиска подстроки

3. Отсутствие вхождений (Рис. 4):

```
crakeny@LAPTOP-TQ9SBN6I:/mnt/c/Users/Hp/OneDrive/Рабочий
xyz
abcdef
-1
```

Рисунок 4. Отсутствие вхождений при поиске

4. Штатный ввод для циклического сдвига (Рис. 5):

```
crakeny@LAPTOP-TQ9SBN6I:/mnt/c/Users/Hp/OneDr
refery
eryref
3
```

Рисунок 5. Демонстрация поиска циклического сдвига

5. Отсутствие циклического сдвига (Рис. 6):

```
crakeny@LAPTOP-TQ9SBN6I:/mnt/c/Users/Hp/OneDrive/Рабочий
klangkarusel
wall
-1
```

Рисунок 6. Отсутствие циклического сдвига

6. Логирование для префикс-функции (Рис. 7):

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
a b b b a c d a a b b a b a c d a b b b


0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 1
    b != a  End of search, length = 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 2
    b != a  End of search, length = 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 3
    b != a  End of search, length = 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 4
    j = 0,→a = a→  searching...
0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 5
    c != a  End of search, length = 0
0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 6
    d != a  End of search, length = 0
0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 7
    j = 0,→a = a→  searching...
0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 8
    j = 0,→a = a→  searching...
0 0 0 0 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 9
    j = 1,→b = b→  searching...
0 0 0 0 1 0 0 1 1 2 0 0 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 10
    j = 2,→b = b→  searching...
0 0 0 0 1 0 0 1 1 2 3 0 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 11
    j = 0,→a = a→  searching...
0 0 0 0 1 0 0 1 1 2 3 1 0 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 12
    j = 1,→b = b→  searching...
0 0 0 0 1 0 0 1 1 2 3 1 2 0 0 0 0 0 0 0 ------------------ prefix-function,→  i = 13
    j = 0,→a = a→  searching...
0 0 0 0 1 0 0 1 1 2 3 1 2 1 0 0 0 0 0 0 ------------------ prefix-function,→  i = 14
    c != a  End of search, length = 0
0 0 0 0 1 0 0 1 1 2 3 1 2 1 0 0 0 0 0 0 ------------------ prefix-function,→  i = 15
    d != a  End of search, length = 0
0 0 0 0 1 0 0 1 1 2 3 1 2 1 0 0 0 0 0 0 ------------------ prefix-function,→  i = 16
    j = 0,→a = a→  searching...
```

Рисунок 7. Логирование для префикс-функции

7. Логирование для поиска подстроки (Рис. 8):

```
АЛГОРИТМ КМП / ПОИСК ПОДСТРОКИ В СТРОКЕ

0 1 2 3 4 5 6 7 8 9 10 11 12
PATTERN:    a b
TEXT:       a b c d a v a b b a b b a


i = 0)      a = a, j->(0) = (j + 1)->(1)

i = 1)      b = b, j->(1) = (j + 1)->(2)
            RESULT += (i - n + 1)->(0), j->(2) = pi[j - 1]->(0)

i = 2)
i = 3)
i = 4)      a = a, j->(0) = (j + 1)->(1)

i = 5)      v != b, j->(1) = pi[j - 1]->(0)

i = 6)      a = a, j->(0) = (j + 1)->(1)

i = 7)      b = b, j->(1) = (j + 1)->(2)
            RESULT += (i - n + 1)->(6), j->(2) = pi[j - 1]->(0)

i = 8)
i = 9)      a = a, j->(0) = (j + 1)->(1)

i = 10)     b = b, j->(1) = (j + 1)->(2)
            RESULT += (i - n + 1)->(9), j->(2) = pi[j - 1]->(0)

i = 11)
i = 12)     a = a, j->(0) = (j + 1)->(1)



ANSWER = 0 6 9
```

Рисунок 8. Логирование для поиска подстроки

8. Логирование для циклических сдвигов (Рис. 9):

```
АЛГОРИТМ КМП / ПОИСК ЦИКЛИЧЕСКОГО СДВИГА

0 1 2 3 4 5
STR A:      r e f e r y
STR B:      e r y r e f


i = 0)
i = 1)      e = e, j->(0) = (j + 1)->(1)

i = 2)      f != r, j->(1) = pi[j - 1]->(0)

i = 3)      e = e, j->(0) = (j + 1)->(1)

i = 4)      r = r, j->(1) = (j + 1)->(2)

i = 5)      y = y, j->(2) = (j + 1)->(3)

i = 6)      r = r, j->(3) = (j + 1)->(4)

i = 7)      e = e, j->(4) = (j + 1)->(5)

i = 8)      f = f, j->(5) = (j + 1)->(6)
j = n, завершение работы

ANSWER: 3
```

Рисунок 9. Логирование для циклического сдвига

Вывод.

В ходе выполнения лабораторной работы был изучен алгоритм нахождения префикс-функции, подстроки в строке (КМП), а также поиск циклического сдвига. Была разработана программа на Python, выполнен анализ сложности алгоритма по времени и памяти, проведено тестирование программы, осуществлено логирование процесса выполнения программы.

Был получен опыт построения алгоритма для решения задачи, а также опыт в сфере оценки и анализа проведенной работы.

ПРИЛОЖЕНИЕ А
КОД ПРОГРАММЫ

Файл KMP.py:

```
import sys


file = "out.txt"

def kmp_prefix_function(s: str):
    n = len(s)
    pi = [0] * n

    f = open(file, "w")
    f.writelines(f'{i} ' for i in range(n))
    f.write('\n')
    f.writelines(f'{i} ' for i in s)
    f.write("\n\n\n")

    for i in range(1, n):
        f.writelines(f'{i} ' for i in pi)
        f.write(" ------------------ prefix-function, \t" + f"i = {i if
n > 1 else '-'}" + "\n")

        j = pi[i - 1]
        while j > 0 and s[i] != s[j]:
            j = pi[j - 1]
        if s[i] == s[j]:
            f.write(f"\t\tj = {j}, \t{s[i]} = {s[j]}\tsearching...\n")
            j += 1
        else:
            f.write(f"\t\t{s[i]} != {s[j]}\tEnd of search, length =
{j}\n")
        pi[i] = j

    f.writelines(f'{i} ' for i in pi)
    f.write(" ------------------ prefix-function, \t" + f"i = {i if n >
1 else '-'}" + "\n")
    f.close()
    return pi

def cyclic_shift_search(A: str, B: str):
    f = open(file, "a")
    f.write("\n\n\t\tАЛГОРИТМ КМП / ПОИСК ЦИКЛИЧЕСКОГО СДВИГА\n\n")

    n = len(A)
    if n != len(B):
        f.write(f"ERROR: len(A) != len(B) ---> {len(A)} != {len(B)}")
        return -1

    if n == 0:
        f.write("Длина строки 0, ответ - 0")
        return 0

    pi = kmp_prefix_function(B)
```

```
    f.write("\t\t\t")
    f.writelines(f'{i} ' for i in range(len(A)))
    f.write('\n')
    f.write("STR A:\t\t")
    f.writelines(f'{i} ' for i in A)
    f.write('\n')
    f.write("STR B:\t\t")
    f.writelines(f'{i} ' for i in B)
    f.write("\n\n\n")

    j = 0
    for i in range(2 * n - 1):
        f.write(f"i = {i})")
        char = A[i % n]
        while j > 0 and char != B[j]:
            f.write(f"\t\t{char} != {B[j]}, j->({j}) = pi[j - 1]->({pi[j
- 1]})\n")
            j = pi[j - 1]
        if char == B[j]:
            f.write(f"\t\t{char} = {B[j]}, j->({j}) = (j + 1)->({j +
1})\n")
            j += 1
        if j == n:
            f.write("j = n, завершение работы")
            f.write(f"\n\nANSWER: {i - n + 1}")
            f.close()
            return i - n + 1
        f.write('\n')

    f.close()
    return -1

def kmp_search(pattern: str, text: str):
    n = len(pattern)
    pi = kmp_prefix_function(pattern)

    result = []
    j = 0

    f = open(file, "a")
    f.write("\n\n\t\tАЛГОРИТМ КМП / ПОИСК ПОДСТРОКИ В СТРОКЕ\n\n")
    f.write("\t\t\t")
    f.writelines(f'{i} ' for i in range(len(text)))
    f.write('\n')
    f.write("PATTERN:\t")
    f.writelines(f'{i} ' for i in pattern)
    f.write('\n')
    f.write("TEXT:\t\t")
    f.writelines(f'{i} ' for i in text)
    f.write("\n\n\n")

    for i in range(len(text)):
        f.write(f"i = {i})")
        while j > 0 and text[i] != pattern[j]:
            f.write(f"\t\t{text[i]} != {pattern[j]}, j->({j}) = pi[j -
1]->({pi[j - 1]})\n")
            j = pi[j - 1]
```

```
        if text[i] == pattern[j]:
            f.write(f"\t\t{text[i]} = {pattern[j]}, j->({j}) = (j + 1)-
>({j + 1})\n")
            j += 1
        if j == n:
            f.write(f"\t\t\tRESULT += (i - n + 1)->({i - n + 1}), j-
>({j}) = pi[j - 1]->({pi[j - 1]})\n")
            result.append(i - n + 1)
            j = pi[j - 1]
        f.write('\n')

    f.write("\n\nANSWER = ")
    f.writelines(f'{i} ' for i in result)
    f.close()

    return result

def ref():
    print("Usage:")
    print("  python program.py                          - обычный поиск
подстроки")
    print("  python program.py -c                  - поиск циклического
сдвига")
    print("  python program.py -h               - показать эту справку")
    print()

if __name__ == "__main__":
    args = sys.argv[1:]
    if "-h" in args:
        ref()
        exit(0)

    P = input().strip()
    T = input().strip()
    if(not P or not T):
        print("EMPTY STRINGS")
        exit(0)

    if not "-c" in args:
        positions = kmp_search(P, T)
        if not positions:
            print(-1)
        else:
            print(",".join(map(str, positions)))
    else:
        positions = cyclic_shift_search(P, T)
        print(positions)
```

Файл tests.py:

```
IMPORT PYTEST
FROM KMP IMPORT *

CLASS TESTPREFIXFUNCTION:
    DEF TEST_EMPTY_STRING(SELF):
        ASSERT KMP_PREFIX_FUNCTION("") == []

    DEF TEST_SINGLE_CHAR(SELF):
```

```
        ASSERT KMP_PREFIX_FUNCTION("A") == [0]

    DEF TEST_ALL_SAME_CHARS(SELF):
        ASSERT KMP_PREFIX_FUNCTION("AAAA") == [0, 1, 2, 3]

    DEF TEST_NO_REPEATS(SELF):
        ASSERT KMP_PREFIX_FUNCTION("ABCDE") == [0, 0, 0, 0, 0]

    DEF TEST_COMPLEX_PATTERN(SELF):
        ASSERT KMP_PREFIX_FUNCTION("ABACAB") == [0, 0, 1, 0, 1, 2]

    DEF TEST_PREFIX_SUFFIX_OVERLAP(SELF):
        ASSERT KMP_PREFIX_FUNCTION("ABCABC") == [0, 0, 0, 1, 2, 3]


CLASS TESTKMPSEARCH:
    DEF TEST_BASIC_SEARCH(SELF):
        RESULT = KMP_SEARCH("AB", "ABAB")
        ASSERT RESULT == [0, 2], F"ОЖИДАЛОСЬ [0, 2], ПОЛУЧЕНО {RESULT}"

    DEF TEST_NO_MATCH(SELF):
        RESULT = KMP_SEARCH("XYZ", "ABCDEF")
        ASSERT RESULT == [], F"ОЖИДАЛСЯ ПУСТОЙ СПИСОК, ПОЛУЧЕНО {RESULT}"

    DEF TEST_PATTERN_AT_END(SELF):
        RESULT = KMP_SEARCH("CD", "ABCD")
        ASSERT RESULT == [2], F"ОЖИДАЛОСЬ [2], ПОЛУЧЕНО {RESULT}"

    DEF TEST_MULTIPLE_MATCHES(SELF):
        RESULT = KMP_SEARCH("AA", "AAAA")
        ASSERT RESULT == [0, 1, 2], F"ОЖИДАЛОСЬ [0, 1, 2], ПОЛУЧЕНО {RESULT}"

    DEF TEST_SINGLE_CHAR_PATTERN(SELF):
        RESULT = KMP_SEARCH("A", "BANANA")
        ASSERT RESULT == [1, 3, 5], F"ОЖИДАЛОСЬ [1, 3, 5], ПОЛУЧЕНО {RESULT}"

    DEF TEST_PATTERN_LONGER_THAN_TEXT(SELF):
        RESULT = KMP_SEARCH("ABCD", "ABC")
        ASSERT RESULT == [], F"ОЖИДАЛСЯ ПУСТОЙ СПИСОК, ПОЛУЧЕНО {RESULT}"

    DEF TEST_EMPTY_TEXT(SELF):
        RESULT = KMP_SEARCH("ABC", "")
        ASSERT RESULT == [], F"ОЖИДАЛСЯ ПУСТОЙ СПИСОК, ПОЛУЧЕНО {RESULT}"


CLASS TESTCYCLICSHIFT:
    DEF TEST_BASIC_CYCLIC_SHIFT(SELF):
        RESULT = CYCLIC_SHIFT_SEARCH("DEFABC", "ABCDEF")
        ASSERT RESULT == 3, F"ОЖИДАЛОСЬ 3, ПОЛУЧЕНО {RESULT}"

    DEF TEST_NO_SHIFT(SELF):
        RESULT = CYCLIC_SHIFT_SEARCH("ABC", "ABC")
        ASSERT RESULT == 0, F"ОЖИДАЛОСЬ 0, ПОЛУЧЕНО {RESULT}"

    DEF TEST_FULL_ROTATION(SELF):
        RESULT = CYCLIC_SHIFT_SEARCH("EABCD", "ABCDE")
        ASSERT RESULT == 1, F"ОЖИДАЛОСЬ 1, ПОЛУЧЕНО {RESULT}"
```

```
    DEF TEST_NOT_CYCLIC_SHIFT(SELF):
        RESULT = CYCLIC_SHIFT_SEARCH("ABC", "XYZ")
        ASSERT RESULT == -1, F"ОЖИДАЛОСЬ -1, ПОЛУЧЕНО {RESULT}"

    DEF TEST_DIFFERENT_LENGTHS(SELF):
        RESULT = CYCLIC_SHIFT_SEARCH("ABC", "ABCD")
        ASSERT RESULT == -1, F"ОЖИДАЛОСЬ -1, ПОЛУЧЕНО {RESULT}"

    DEF TEST_EMPTY_STRINGS(SELF):
        RESULT = CYCLIC_SHIFT_SEARCH("", "")
        ASSERT RESULT == 0, F"ОЖИДАЛОСЬ 0, ПОЛУЧЕНО {RESULT}"

    DEF TEST_SINGLE_CHAR(SELF):
        RESULT = CYCLIC_SHIFT_SEARCH("A", "A")
        ASSERT RESULT == 0, F"ОЖИДАЛОСЬ 0, ПОЛУЧЕНО {RESULT}"

    DEF TEST_PERFORMANCE_ARRAY(SELF):
        BASE = "ABCDEFGHIJ"
        FOR SHIFT IN RANGE(LEN(BASE)):
            SHIFTED = BASE[SHIFT:] + BASE[:SHIFT]
            RESULT = CYCLIC_SHIFT_SEARCH(SHIFTED, BASE)
            ASSERT RESULT == (LEN(BASE) - SHIFT) % LEN(BASE), \
                F"СДВИГ {SHIFT}: ОЖИДАЛОСЬ {(LEN(BASE)-SHIFT) % LEN(BASE)},
ПОЛУЧЕНО {RESULT}"

IF __NAME__ == "__MAIN__":
    PYTEST.MAIN([__FILE__, "-V"])
```